COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S3 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/21/2026	**DATE SUBMITTED:**	7/28/2026
NAME:	Gabriel Immanuel Serrano	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 3.1 LINKED LISTS

1. PROCEDURE OUTPUT

Screenshot

main.cpp > main()

```cpp
#include <iostream>
#include "simply_LL.h"

int main(){
    //  creating an empty list also allocate memory
    SimplyList<char>* head= new SimplyList<char>; //x1001
    SimplyList<char>* second= new SimplyList<char>; //x1002
    SimplyList<char>* third= new SimplyList<char>; //x1003
    SimplyList<char>* fourth= new SimplyList<char>; //x1004
    SimplyList<char>* fifth= new SimplyList<char>; //x1005

    //assign the data
    head->data = 'G'; // the head has the J letter now
    head->next = second; // the next address

    second->data = 'A';
    second->next = third;

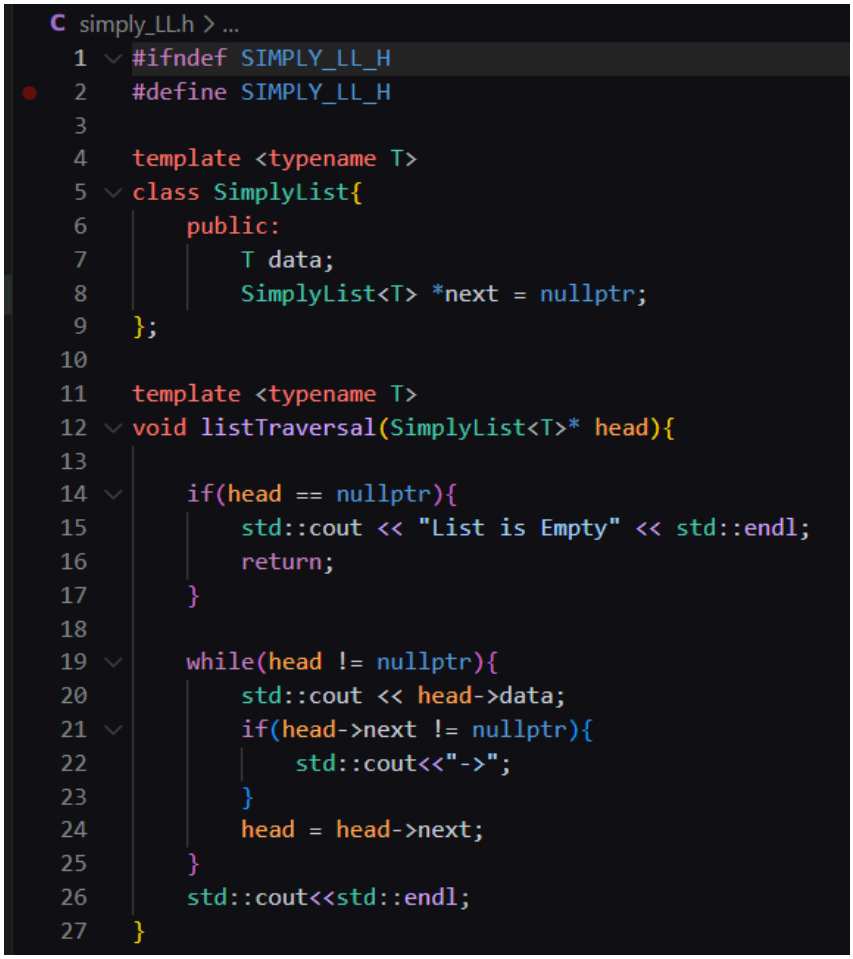
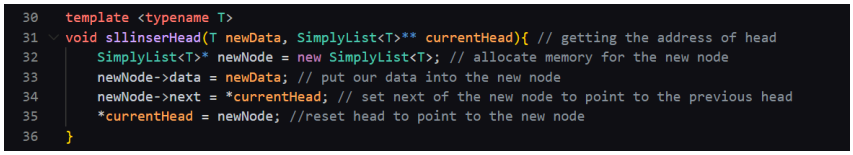
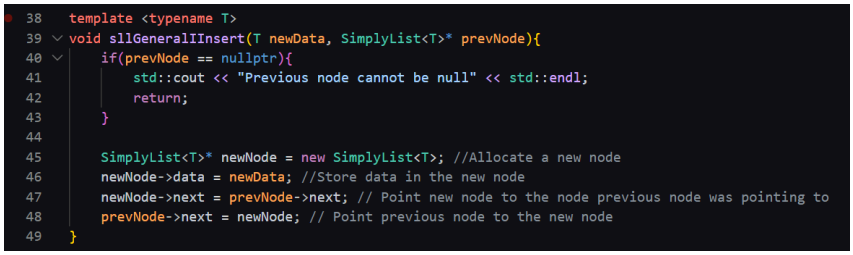
    third->data = 'B';
    third->next = fourth;

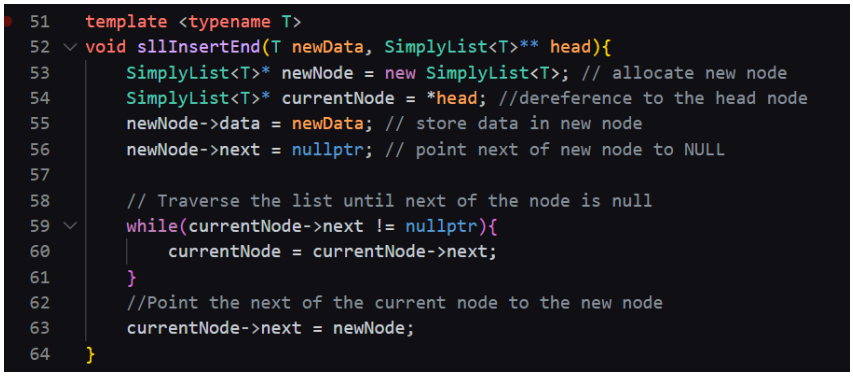
    fourth->data = 'Q';
    fourth->next = nullptr;
```

Discussion

The program creates a singly linked list by dynamically allocating memory for five nodes. Each node stores a character and is connected to the next node using the next pointer. But on the program we only use 4 nodes, the 4 nodes are assigned with the character G,A,B, and Q, and their next pointer connected to the next node. Then the last node is pointed to nullptr. This code doesn't print the text because it only stores the character data inside the node, but by using a traversal you can print out the characters inside the node.

Table 3-1. Output of Initial/Simple Implementation

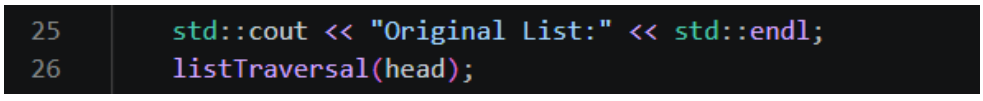
Operation	Screenshot
Traversal	
Insertion at head	
Insertion at any part of the list	

Insertion at the end	
Deletion of node	

```
51  template <typename T>
52  void sllInsertEnd(T newData, SimplyList<T>** head){
53      SimplyList<T>* newNode = new SimplyList<T>; // allocate new node
54      SimplyList<T>* currentNode = *head; //dereference to the head node
55      newNode->data = newData; // store data in new node
56      newNode->next = nullptr; // point next of new node to NULL
57
58      // Traverse the list until next of the node is null
59      while(currentNode->next != nullptr){
60          currentNode = currentNode->next;
61      }
62      //Point the next of the current node to the new node
63      currentNode->next = newNode;
64  }
```

```
66  template <typename T>
67  void sllDeleteNode(T findData, SimplyList<T>** head){
68      //check if there is no list to be deleted
69      if (*head == nullptr){
70          return;
71      }
72
73      SimplyList<T>* currNode = *head;
74      SimplyList<T>* prevNode = nullptr;
75
76      //searching loop need both to be true
77      while(currNode != nullptr && currNode->data != findData){
78          prevNode = currNode;
79          currNode = currNode->next;
80      }
81
82      //data was mot found
83      if(currNode == nullptr){
84          return;
85      }
86
87      // what if the head node need to be deleted
88      if(prevNode == nullptr){
89          *head = currNode->next;
90      }else{
91          prevNode->next = currNode->next;
92      }
93
94      // we have to delete the node?
95      delete currNode;
96
97  }
```

Table 3-2. Code for the List Operations

a.	**Source Code**	
	Console	

```
25      std::cout << "Original List:" << std::endl;
26      listTraversal(head);
```

```
Original List:
G->A->B->Q
```

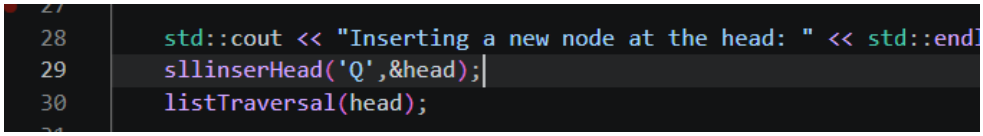
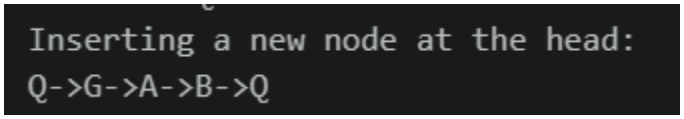
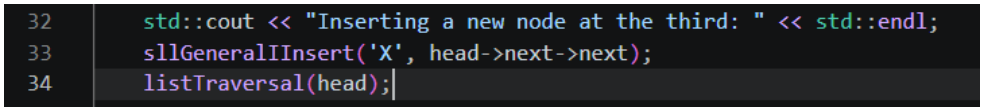
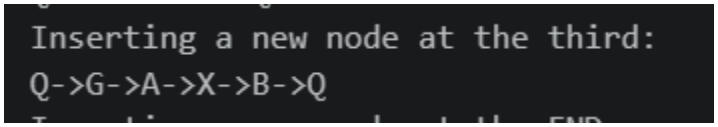
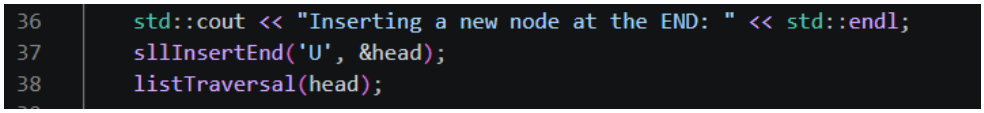
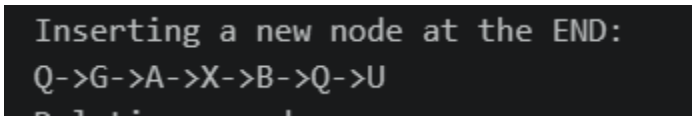
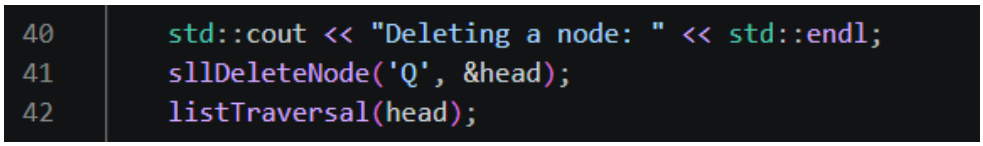
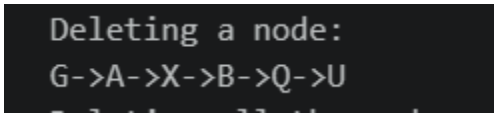
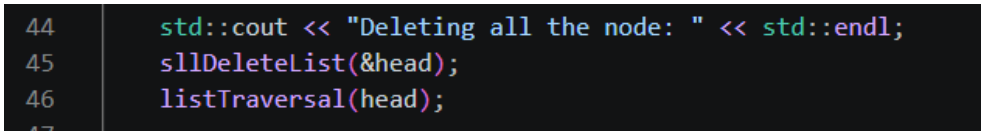
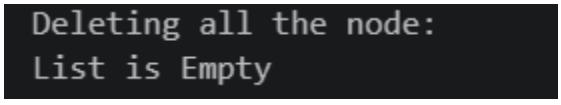
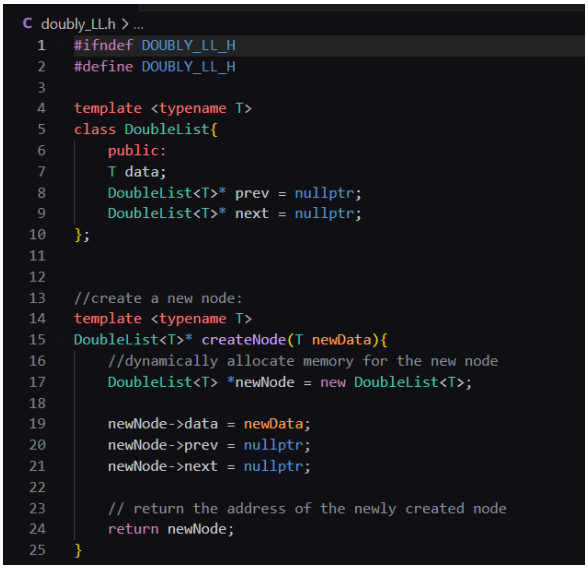
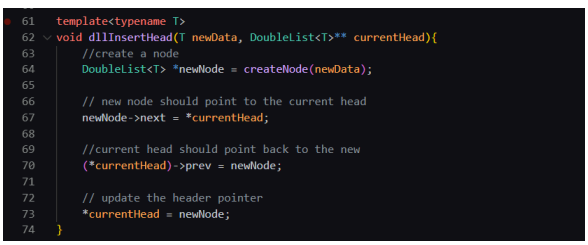
b.	**Source Code**	
	Console	
c.	**Source Code**	
	Console	
d.	**Source Code**	
	Console	
e.	**Source Code**	
	Console	
f.	**Source Code**	
	Console	

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots(s)	Analysis
	**createNode :** The createNode() function is used to create a new node in the doubly linked list. It creates a new node that stores the given data in the data variable, and it sets both the prev and next pointers to nullptr because the new node that we created is not yet connected to the existing nodes. Then, the function returns the address of the new node.
	**dllTraverse:** The dllTraverse() function is used to display the elements on the linked list in forward and backward. It works by checking if the list is empty on the currentNode is equal to nullptr. If the list is not empty, it prints the node from head to tail by moving the next node using the next pointer. Now after reaching the end of the list, the function will then print the node in reverse starting from the tail to the head by using the prev node and prev pointer.
	**dllInsertHead:** The dllInsertHead function is used to insert a new node at the beginning by creating a new node using the createNode function. Then, we connect the new node to the current head. After that, the prev pointer of the current head is connected to the new node. Then, the new node is assigned as the current head.

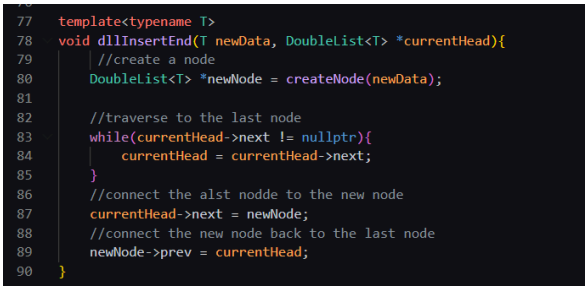
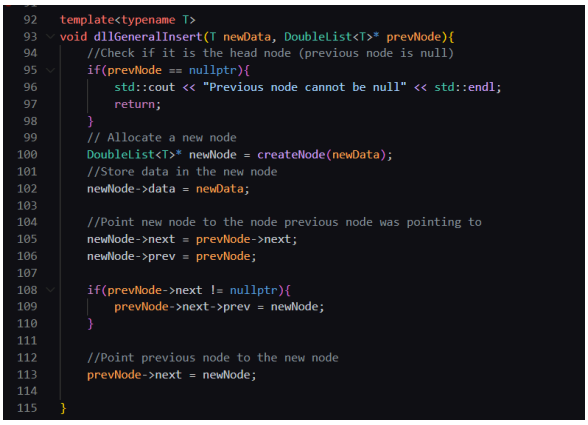
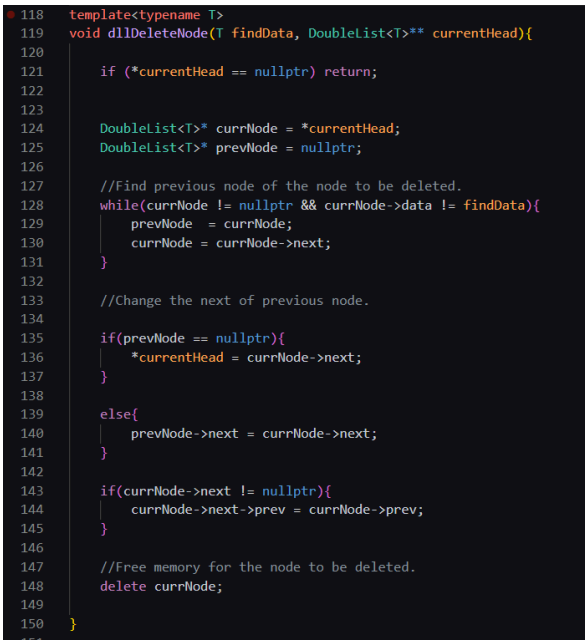
	**dllInsertEnd:** The dllInsertEnd() function is used to insert a new node at the end. It creates a new node by using the createNode function. Then, it traverses the list by moving the pointer until it reaches the tail or the last node. Instead of the last node pointing to nullptr, its next pointer is connected to the new node. After that, the prev pointer of the new node is connected back to the prev last node.
	**dllGeneralInsert:** The dllGeneralInsert() function is used to insert a new node after a selected prev node. This function checks if the prev node is equal to nullptr. If the prev node is valid, the function creates a new node and connects its next pointer to the node after the prev node. The prev pointer of the new node is then connected to the previous node.
	**dllDeleteNode:** The dllDeleteNode() function is used to find and delete a certain node. It works by checking if the list is empty. If the list is not empty, the function goes through the list to find the specific node. After finding the node, the next pointer of the previous node skips the node that we are trying to delete. Then the prev pointer of the next node points backward to connect the node in the list, then it deletes the node we selected.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Playlist.h

Playlist.h > ...

```h
#ifndef PLAYLIST_H
#define PLAYLIST_H

#include<iostream>
#include<string>

class Song{
    public:
    std::string data;
    Song* prev = nullptr;
    Song* next = nullptr;
};

// CREATE A SONG
Song* createSong(std::string newData){
    Song *newSong = new Song;
    newSong->data = newData;
    newSong->prev = newSong;
    newSong->next = newSong;
    return newSong;
}

// ADD A SONG
void playlistInsertHead(std::string newData, Song** currentHead){
    Song *newSong = createSong(newData);

    if(*currentHead == nullptr){
        *currentHead = newSong;
        return;
    }
    Song *tail = (*currentHead)->prev;

    newSong->next = *currentHead;
    newSong->prev = tail;
    (*currentHead)->prev = newSong;
    tail->next = newSong;
    *currentHead = newSong;
}
```

\

```cpp
// DELETE A SONG
void playlistDeleteNode(std::string findData, Song** currentHead){

    if (*currentHead == nullptr) return;

    Song* currNode = *currentHead;

    do{
        if(currNode->data == findData){

            if(currNode->next == currNode){
                *currentHead = nullptr;
            }
            else{
                currNode->prev->next = currNode->next;
                currNode->next->prev = currNode->prev;

                if(currNode == *currentHead){
                    *currentHead = currNode->next;
                }
            }

            delete currNode;
            return;
        }
        currNode = currNode->next;
    } while(currNode != *currentHead);

    std::cout << "Song was not found in playlist" << std::endl;
}
```

```cpp
// PRINT SONGS
void playlistTraverse(Song *currentHead){

    if(currentHead == nullptr){
        std::cout << "The playlist is empty, Please add a song" << std::endl;
        return;
    }
    Song *currentNode = currentHead;
    int track = 1;

    std::cout << "Now Playing: " << std::endl;

    do{
        std::cout << track << ". " << currentNode->data << std::endl;
        currentNode = currentNode->next;
        track++;
    } while(currentNode != currentHead);

    std::cout << std::endl;
}

#endif
```

Main.py

spotify.cpp > ...

```cpp
#include "playlist.h"
#include <iostream>
#include <string>

int main(){
    Song *playlist = nullptr;

    std::cout << "===== CURRENT PLAYLIST =====" << std::endl;
    playlistInsertHead("Sparkle | Your Name AMV", &playlist);
    playlistInsertHead("I REALLY WANT TO STAY AT YOUR HOUSE", &playlist);
    playlistInsertHead("Color Your Night", &playlist);
    playlistTraverse(playlist);


    std::cout << "\n===== ADDING SONG =====" << std::endl;
    playlistInsertHead("Wakeshima Kanon - Tsukinami", &playlist);
    playlistTraverse(playlist);


    std::cout << "\n===== REMOVING A SONG =====" << std::endl;
    playlistDeleteNode("Color Your Night", &playlist);
    playlistTraverse(playlist);


    std::cout << "\n===== PLAYLIST =====" << std::endl;
    Song *current = playlist;
    std::cout << "Current Song: " << current->data << std::endl;
    current = current->next;
    std::cout << "Next Song: " << current->data << std::endl;
    current = current->prev;
    std::cout << "Previous Song: " << current->data << std::endl;

    return 0;
}
```

OUTPUT:

```
===== CURRENT PLAYLIST =====
Now Playing:
1. Color Your Night
2. I REALLY WANT TO STAY AT YOUR HOUSE
3. Sparkle | Your Name AMV


===== ADDING SONG =====
Now Playing:
1. Wakeshima Kanon - Tsukinami
2. Color Your Night
3. I REALLY WANT TO STAY AT YOUR HOUSE
4. Sparkle | Your Name AMV


===== REMOVING A SONG =====
Now Playing:
1. Wakeshima Kanon - Tsukinami
2. I REALLY WANT TO STAY AT YOUR HOUSE
3. Sparkle | Your Name AMV


===== PLAYLIST =====
Current Song: Wakeshima Kanon - Tsukinami
Next Song: I REALLY WANT TO STAY AT YOUR HOUSE
Previous Song: Wakeshima Kanon - Tsukinami
```

Code analysis:

This supplementary activity uses a header file to create a circular doubly linked list. I Initialized a song class that contains string data variables that can store the name of the song, a prev pointer that connects the previous song, and a next pointer that points to the next song. We then created a createSong() function, this function is used to create a new song or a new node by storing the given song name inside the data variable. Then the prev and the next pointer are both connected to the new song so that the song can work as a circular linked list. The playlistInsertHead() function is used to add a new song at the beginning of the playlist, it has a condition that if a playlist is empty, the new song will become the head, and if the playlist already contain a song, the new song is connected to the head and the tail of the current node, which then becomes the current head. The playlistDeleteNode() function is used to search for and delete a selected song. It traverses the playlist until it finds the song, then it connects the prev and next song to remove the song from the playlist. Lastly the playlistTraverse() function, has a catch function checking if the currentHead is null if not it will proceed to display all the songs in the playlist. It starts from the head and goes to each of the nodes until it reaches the tail , and it loops back until it reaches the head again.After this we implemented into the spotify.cpp file to test the function of it all, we created a pointer that is named playlist so that we can use this as the base for adding songs. Then we test each function of inserting a song, adding a song at the head, deleting a song, and showing the previous, current , and next song that will be played.

Conclusion:
In this activity, I learned how to create, insert, delete, and traverse nodes in singly and doubly linked lists. The procedure helped me understand how the next and prev pointers connect the nodes. However, in the supplementary activity, I relied a little on AI because I did not know how to implement a circular linked list in the code. It was very challenging, to say the least, but I managed to complete it with the help of AI. However, I learned the concepts and how the circular linked list is supposed to work instead of heavily relying on AI. Because of that, I was able to create a playlist that can add, delete, and display songs. Overall, I think I did okay with the singly and doubly linked lists, but I still had difficulty with the circular linked list. I hope to understand more about how to create and implement a circular linked list in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction and Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammarly was used for Grammar correction while ChatGPT was used in order to understand how Circular Linked list work and how to implement it on the supplementary activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.